Final Paper: Research Topic Guide

Oct 27, 2026 · @Victor Preciado

How to use this guide

If you do not have a project topic yet, choose one of the topics below, adapt it to make it your own, and turn it into a paper. The paper format will be released soon.

The project rules shape every topic (Canvas has the current version):

- Groups of up to 3.
- The dataset must be public and already usable: no data collection, no scraping. Synthetic data from a simulation you write yourself counts when you need the ground truth to evaluate your work.
- Novelty is very welcome but not required. A careful replication, a fair comparison, or a known method on new data makes a good project.
- Your baseline should use material taught before the draft is due. Later material (trees, neural networks, PCA) can be the comparison.
- AI assistants are encouraged at every stage, and the paper's acknowledgements include an AI use statement.

Each entry below provides a feasible research question, a first experiment, the data, the course sections it builds on, and something to read. The level is a guide: **Accessible** means course tools reach a clean result; **Moderate** needs one new method or library; **Ambitious** needs both, plus a fallback plan. You may combine two topics or propose your own, as long as it passes the test in the next section.

#	Topic	Builds on	Data	Level
1	Who gets to be vaccinated?	1C, 1D, 2B	SocioPatterns contacts, simulation	Moderate
2	The epidemic threshold from local structure	2B, 4A, 5A, 5B	Netzschleuder, random graphs	Accessible
3	Measuring a network you cannot see whole	1C, 1D, 4C	SNAP social networks	Accessible

#	Topic	Builds on	Data	Level
4	Forecasting an outbreak	3C, 4A, 4C, 7B	OWID COVID-19, CDC ILINet	Moderate
5	Where to test on a budget	1D, 3D, 5C	SocioPatterns, simulation	Ambitious
6	Persuasion in contested networks	2B, 7A, 9G to 9J	SNAP, karate club	Moderate
7	Recovering a network from its dynamics	3D, 4A, 5B	simulation on real topologies	Moderate
8	Finding communities	9A, 9G to 9J, 8A	block models, Cora	Moderate
9	How far can a linear model go?	2B, 4A, 5B, 9A	simulated oscillators	Moderate
10	Discovering equations with the lasso	1C, 4A, 5A, 5B	simulated Lorenz, Van der Pol	Accessible
11	Latent dynamics of brain signals	4A, 9A, 9B	PhysioNet EEG	Ambitious
12	Learning a solution operator	5B, 8A, 9A, 9B	simulated PDEs, PDEBench	Ambitious
13	Forecasting ride demand across a city	4A, 5B, 7B	NYC TLC trip records	Moderate
14	Low test error, unstable controller?	4A, 7A, 7B, 8A	Gymnasium, simulation	Moderate
15	Learning the region of attraction	3A to 3D, 4B, 8A	simulation	Accessible
16	How fragile is a neural network?	5B, 8A, 8B	MNIST, Fashion-MNIST	Moderate
17	Where can a learned model go?	1C, 4C, 8A	simulation	Ambitious
18	Optimizers as dynamical systems	2B, 3C, 5B	synthetic, UCI	Accessible

#	Topic	Builds on	Data	Level
19	Learning together over a network	2B, 3C	UCI, MNIST split across nodes	Moderate
20	Do the probabilities mean what they say?	3C to 3E, 4B	UCI, OpenML	Accessible
21	A rare positive class	3D, 4B, 5B	OpenML credit-card fraud	Accessible
22	Honest error bars	1C, 1D, 4A to 4C	simulation, UCI	Accessible
23	Trees or neural networks for tables?	4A, 7A, 7B, 8A	OpenML benchmark	Moderate
24	Double descent	2B, 5B, 8B	MNIST, synthetic	Moderate
25	Which inputs really matter?	1C, 5A, 5B, 7B	UCI, OpenML	Accessible

Key dates

The project runs in six milestones, from the proposal to the final paper and video. Each is submitted on Canvas and due at 11:59 PM Eastern time.

Milestone	Day	Due
Proposal	Monday	Nov 2, 2026
Baseline checkpoint	Monday	Nov 9, 2026
Early paper draft	Monday	Nov 16, 2026
Analysis complete	Monday	Nov 30, 2026
Paper and video draft	Friday	Dec 4, 2026
Final paper and five-minute video	Friday	Dec 11, 2026

What makes a topic work

A good topic is one question that a figure can answer, measured against something you trust. Before you commit, check that your idea has all four of these:

1. **One sharp question with a measurable answer.** "Does method A beat method B on this data, by how much, and when does the gap close?"
2. **Something to check against.** The best is a simulation where you know the truth, as the course's answer key does. Otherwise, a test set you touch once, at the end.
3. **A baseline from the course.** Logistic regression, ridge, a tuned K-fold CV. A sophisticated method never compared with a simple one tells the reader nothing.
4. **Honest uncertainty.** Error bars from cross-validation, the bootstrap, or repeated runs over seeds. A difference smaller than its error bar is a tie, and the paper says so.

Too broad	Right-sized
Use deep learning to predict COVID-19	Does a fitted SIR model forecast weekly cases two weeks ahead better than gradient boosting on lagged cases, across ten US states?
Study the robustness of neural networks	How does weight decay change the Lipschitz bound and the adversarial accuracy of a two-layer network on MNIST?
Analyze social networks	How many sampled ego-networks do we need to estimate a network's spectral radius within 10%?

A paper can take one of three shapes. A **replication** reproduces a published figure and then stresses it (more noise, less data, a new dataset). A **comparison** runs several methods on several datasets with equal tuning effort. An **extension** applies a known method to a setting its authors did not try.

The most common ways a project fails are all from Module 4: tuning on the test set, preprocessing on all the data before splitting, shuffling time series into random folds, and reporting the best of many seeds. Every step learned from data goes inside the folds.

Using AI well

Use AI as a fast collaborator, never as a source: every number, citation and claim in your paper must be something you ran, opened or checked.

Where it helps most:

- **Narrowing a topic.** Ask for three right-sized questions built on one entry below, then choose and sharpen one yourselves.
- **Getting oriented in a new literature.** Ask for the key papers and terms, then find and open each one.
- **Code.** Simulators, data loading, plotting, debugging. Read every line you keep.
- **A critical reviewer.** Paste your methods section and ask where the validation could leak, or where a claim outruns the evidence.
- **Writing.** Tightening prose and fixing LaTeX.

What you must verify yourselves:

- **Every reference exists and says what you cite it for.** Assistants invent plausible papers by real authors. A fabricated reference is a serious error, whoever typed it.
- **Every number comes from code you ran**, with fixed seeds, that a teammate can rerun from scratch.
- **The code does what you think.** Generated code often leaks the test set (scaling before splitting) or silently drops rows.
- **Every member can explain every equation and figure**, in the paper and in the video.

Keep a short log of which tools you used and for what. The AI use statement in your acknowledgements summarizes it, for example:

> We used [assistant] to draft the simulation code in Section 3 and to suggest related work. We verified every reference and reran every experiment ourselves, and used it to edit the prose for clarity. The analysis and conclusions are our own.

Theme 1: Networks, epidemics and spreading

Epidemics, rumors and opinions spread over networks, and the structure of the network decides how far they go. A good first read for topics 1 to 7 is the survey by Nowzari, Preciado and Pappas, Analysis and Control of Epidemics: A Survey of Spreading Processes on Complex Networks (IEEE Control Systems Magazine, 2016).

1. Who gets to be vaccinated?

Question. With a fixed number of doses on a real contact network, how much does a targeted allocation shrink outbreaks compared with random or high-degree choices?

Try first. Simulate a stochastic SIS or SIR epidemic on a SocioPatterns contact network. Compare random, degree, eigenvector centrality, and a greedy rule that removes the node that most lowers the spectral radius. Report outbreak size with bootstrap intervals over a few hundred runs per strategy.

Go further. Solve the convex allocation of the first paper below with CVXPY's geometric programming mode, and look for a network where greedy centrality fails.

Read. Preciado et al., Optimal Resource Allocation for Network Protection Against Spreading Processes (IEEE TCNS, 2014); Zargham and Preciado, Worst-Case Scenarios for Greedy, Centrality-Based Network Protection Strategies (CISS, 2014).

2. The epidemic threshold from local structure

Question. An SIS epidemic dies out when the infection rate is below a threshold set by the spectral radius, a global property of the network. How well can cheap local counts (degree moments, triangles, small motifs) predict it?

Try first. Collect a few hundred networks from Netzschleuder plus random graphs from NetworkX. Compute the spectral radius and a dozen local features, then fit linear regression and the lasso on a log scale. Validate with folds grouped by network family, so the model is tested on kinds of networks it has not seen.

Go further. Compare your predictions with the moment-based bounds in the papers below. Does the lasso keep the degree and triangle features those papers single out?

Read. Preciado and Jadbabaie, Moment-Based Spectral Analysis of Large-Scale Networks Using Local Structural Information (IEEE/ACM ToN, 2013); Chen, Ogura and Preciado, Bounds on the Spectral Radius of Digraphs from Motif Counts (SIAM J. Matrix Anal. Appl., 2020); Barreras, Hayhoe, Hassani and Preciado, Measure-Theoretic Bounds on the Spectral Radius of Graphs (Linear Algebra Appl., 2021).

3. Measuring a network you cannot see whole

Question. If you can only sample nodes and their neighborhoods (ego-networks), how many samples estimate the network's average degree, clustering and spectral radius within 10%, and are your error bars honest?

Try first. Take a large network from SNAP, whose full structure is your answer key. Sample ego-networks uniformly and by random walk, and reweight the walk, which over-samples hubs. Measure bias and spread over many repeated samples, and check how often bootstrap intervals contain the truth.

Go further. Estimate the spectral radius from the sampled ego-networks with the moment bounds of topic 2.

Read. Wu and Preciado, Laplacian Spectral Properties of a Graph from Random Structural Samples (SIAM SDM, 2014); Preciado and Ogura, Structural Analysis of Spreading Processes from Ego-Nets (IFAC NecSys, 2016).

4. Forecasting an outbreak

Question. Does a fitted compartmental model (SIR or SEIR with time-varying transmission) forecast weekly cases better than ridge or gradient boosting on lagged counts, and at which horizons?

Try first. Use COVID-19 cases from Our World in Data or US influenza-like illness from CDC ILINet through Delphi Epidata. Evaluate on a rolling origin: train up to week t, forecast weeks t+1 to t+4, never shuffle. Always include the persistence forecast (next week equals this week).

Go further. Fit several regions jointly with shared parameters, as the first paper below does, and check the coverage of your prediction intervals.

Read. Hayhoe, Barreras and Preciado, Multitask Learning and Nonlinear Optimal Control of the COVID-19 Pandemic: A Geometric Programming Approach (Annual Reviews in Control, 2021); Alamo et al., Data-Driven Methods for Present and Future Pandemics: Monitoring, Modeling and Managing (Annual Reviews in Control, 2021).

5. Where to test on a budget

Question. With k tests per day on a contact network, which allocation detects a new outbreak earliest?

Try first. Simulate outbreaks seeded at random on a SocioPatterns network. Compare testing random people, high-degree people, contacts of past positives, and the people with the highest estimated probability of infection, estimated by Monte Carlo. Report the distribution of time to first detection.

Go further. Make the tests imperfect, with a given sensitivity and specificity, and weigh missed cases against false alarms with costs, as in Section 3D.

Read. Batlle, Fernandez-Granda, Bruna and Preciado, Adaptive Test Allocation for Outbreak Detection and Tracking in Contact Networks (SIAM J. Control Optim., 2022).

6. Persuasion in contested networks

Question. Two campaigns each choose k people to persuade. Under a linear opinion dynamics model, which seeds win, and can a model learn a person's influence from simple features?

Try first. Implement the Friedkin-Johnsen model on SNAP's email-Eu-core network or the karate club graph; each person's influence then has a closed form. Compare greedy, top-degree and best-response seeding. Then predict influence from degree, centrality and community with a regression tree and a random forest.

Go further. Replace opinions with two competing contagions that spread at once.

Read. Tumu, Vasile, Preciado and Mangharam, Multiagent Social Influence: Modeling Persuasion in Contested Social Networks (ACC, 2026); Eshghi et al., Spread, then Target, and Advertise in Waves: Optimal Capital Allocation Across Advertising Channels (IEEE TNSE, 2020); Watkins, Nowzari, Preciado and Pappas, Optimal Resource Allocation for Competitive Spreading Processes on Bilayer Networks (IEEE TCNS, 2018).

7. Recovering a network from its dynamics

Question. Given only time series of node states, can you recover who influences whom, and how much data does it take?

Try first. Simulate a noisy linear diffusion on a known sparse directed graph. Regress each node's next state on every node's current state with the lasso, and score the recovered edges with precision-recall curves: edges are rare, so the ROC flatters you (Section 3D). Vary the series length and the noise.

Go further. Nonlinear dynamics, such as coupled oscillators or an SIS epidemic, where a linear regression is misspecified.

Read. Shahrampour and Preciado, Topology Identification of Directed Dynamical Networks via Power Spectral Analysis (IEEE TAC, 2015); Shahrampour and Preciado, Reconstruction of Directed Networks from Consensus Dynamics (ACC, 2013); Becker et al., Identification of Networks of Wilson-Cowan Neuronal Oscillators by Inverse Sigmoidal Transformations (IEEE SPMB, 2016).

8. Finding communities

Question. When does spectral clustering recover planted communities that K-means on raw adjacency rows misses, and where does every method fail?

Try first. Generate stochastic block models with two to four communities, narrowing the gap between within-group and between-group edge probabilities. Compare K-means on adjacency rows with K-means on the leading eigenvectors of the adjacency and Laplacian matrices (Sections 9A and 9H), scored against the planted labels.

Go further. Sparse graphs, where the non-backtracking matrix helps, and the Cora citation network from PyTorch Geometric against a graph neural network.

Read. Krzakala et al., Spectral redemption in clustering sparse networks (PNAS, 2013); Masuda, Preciado and Ogura, Analysis of Epidemics in Networks via the Non-Backtracking Matrix (IMA J. Appl. Math., 2020); Hayhoe, Riess, Preciado and Ribeiro, Transferable Hyper-graph Neural Networks via Spectral Similarity (PMLR, 2023).

Theme 2: Learning and controlling dynamical systems from data

Pendulums, cities and brains are often known only through measurements. These topics ask how much of a system's behavior a model learned from data captures, how far ahead it can be trusted, and what happens when it is used to act. Simulators give you the answer key for free.

9. How far can a linear model go?

Question. Koopman theory says a nonlinear system evolves linearly on suitable functions of its state. How far ahead does a linear model on lifted features predict a nonlinear system, compared with a small neural network?

Try first. Simulate a damped pendulum, the Duffing oscillator and the Van der Pol oscillator with measurement noise. Fit DMD (least squares on the raw state), EDMD with polynomial or radial basis features plus a ridge penalty, and an MLP that predicts the next state. Plot forecast error against horizon on held-out trajectories, choosing the dictionary size and the penalty by cross-validation over trajectories. PyDMD and PyKoopman implement the first two.

Go further. The Lorenz system, where every model eventually loses: measure how long each one stays useful.

Read. Williams, Kevrekidis and Rowley, A Data-Driven Approximation of the Koopman Operator: Extending Dynamic Mode Decomposition (J. Nonlinear Sci., 2015); Preciado, Koopman Generator Decomposition for Port-Hamiltonian Systems (under review); Chen, Chen and Preciado, Safety Verification of Nonlinear Autonomous Systems via Occupation Measures (CDC, 2019).

10. Discovering equations with the lasso

Question. Sparse identification of nonlinear dynamics (SINDy) regresses measured derivatives on a library of candidate terms with a sparsity penalty. How much noise, and how little data, can it take before it finds the wrong equations?

Try first. Simulate the Lorenz or Van der Pol system, estimate derivatives by finite differences, and regress them on a polynomial library with the lasso and with PySINDy's thresholded least squares. Score the recovered terms with precision and recall against the true equations. Compare the penalty cross-validation picks with the one that recovers the truth.

Go further. Bootstrap the data to see which terms are selected stably (Section 1C), and try best subset selection on a small library (Section 5A).

Read. Brunton, Proctor and Kutz, Discovering governing equations from data by sparse identification of nonlinear dynamical systems (PNAS, 2016); Becker and Preciado, Identification of State-Space Linear Time-Varying Systems with Sum-of-Norms Regularization (ACC, 2018).

11. Latent dynamics of brain signals

Question. How many hidden states does a linear dynamical model need to predict multichannel EEG, and does it beat PCA followed by an autoregression?

Try first. Use the PhysioNet EEG Motor Movement/Imagery dataset (64 channels, 109 subjects). Compare a per-channel autoregression, PCA to r components followed by a vector autoregression (Sections 9A and 9B), and a state-space model of order r fitted by subspace identification. Choose r on held-out time blocks, never random folds, and report prediction error across subjects with intervals.

Go further. Do the fitted model's states classify the imagined movement better than raw features?

Read. Becker, Bassett and Preciado, Large-Scale Dynamic Modeling of Task-fMRI Signals via Subspace System Identification (J. Neural Eng., 2018); Becker and Preciado, Blind Subspace System Identification via Riemannian Optimization (ACC, 2017).

12. Learning a solution operator

Question. Neural operators learn maps between functions, such as from an initial condition to the solution of a PDE at a later time. How does a deep operator network compare with the course's own PCR, which compresses inputs and outputs with PCA and

regresses between the coefficients?

Try first. Generate pairs from the 1-D heat or Burgers′ equation with random initial conditions and a simple finite-difference solver, or download them from PDEBench. Fit PCR (Section 9B), its ridge version, an MLP on the PCA coefficients, and a DeepONet. Plot error against the number of training pairs and the number of components.

Go further. Test on initial conditions rougher than any seen in training, and see which model degrades gracefully.

Read. Kissas et al., Learning Operators with Coupled Attention (JMLR, 2022); Lu et al., Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators (Nature Machine Intelligence, 2021).

13. Forecasting ride demand across a city

Question. Does knowing what happens in neighboring zones improve hourly forecasts of taxi and ride-hail pickups?

Try first. Download a few months of NYC TLC trip records and count pickups by taxi zone and hour. Compare a seasonal naive forecast (same hour last week), ridge on a zone′s own lags, ridge on its own and its neighbors′ lags, and gradient boosting. Validate in time order, and map where the neighbors help.

Go further. Turn the forecasts into a dispatch decision: how many vehicles to send to each zone, and what a forecast error costs.

Read. Chen, Miao, Pappas and Preciado, Hierarchical Data-Driven Vehicle Dispatch and Ride-sharing (CDC, 2017).

14. Low test error, unstable controller?

Question. When a model learns a controller by imitating an expert, does a lower test error mean a safer closed loop?

Try first. Take a pendulum with a simple expert controller (for example, LQR near the upright position) and record state and action pairs. Imitate it with a regression tree, a random forest, gradient boosting and an MLP (Sections 7A, 7B and 8A). For each, measure the test MSE on held-out states and the success rate in closed loop from random starts the expert itself can stabilize, and plot one against the other. Gymnasium′s Pendulum or your own ODE solver both work.

Go further. Ask the expert to label the states the learned controller actually visits, retrain, and repeat.

Read. Aydinoglu, Becker and Preciado, Data-Driven Stabilization of Nonlinear Systems via Tree-Based Ensemble Learning (CDC, 2019).

Theme 3: Robust, safe and well-trained models

Learned models now steer cars and robots, so accurate on average is not enough: we also want to know where a model can go wrong and how far its output can move. These topics bring the course's classification and optimization tools to that question.

15. Learning the region of attraction

Question. Which starting states of a nonlinear system settle at a stable equilibrium? Label simulated starting states by where they end up, and learn the boundary as a classification problem.

Try first. Use the Van der Pol oscillator run backward in time, whose stable origin has a bounded basin, or a damped pendulum. Compare logistic regression with polynomial features (Section 3C), kNN, a random forest and an MLP, scored against a dense simulated answer key as N grows. Calling an unsafe state safe is the worse mistake, so pick the threshold with costs (Section 3D) and report the rate of false "safe" calls.

Go further. Fit a quadratic Lyapunov function and compare its certified region with your classifier's: guaranteed but conservative, against accurate but unguaranteed.

Read. Chen, Fazlyab, Morari, Pappas and Preciado, Learning Region of Attraction for Nonlinear Systems (CDC, 2021); the same authors, Learning Lyapunov Functions for Hybrid Systems (HSCC, 2021).

16. How fragile is a neural network?

Question. How far can a small change to the input move a network's output, and does weight decay (Section 5B) make a network harder to fool?

Try first. Train MLPs on MNIST or Fashion-MNIST at several weight-decay values. For each, compute a cheap upper bound on the Lipschitz constant (the product of the layers' spectral norms), a lower bound (the largest input gradient over test images), and the accuracy under the fast gradient sign attack at several attack sizes. Plot robust accuracy against the bound.

Go further. Tighten the bound with LipSDP's semidefinite program, and compare adversarial training with weight decay.

Read. Fazlyab, Robey, Hassani, Morari and Pappas, Efficient and Accurate Estimation of Lipschitz Constants for Deep Neural Networks (NeurIPS, 2019); Goodfellow, Shlens and Szegedy, Explaining and Harnessing Adversarial Examples (ICLR, 2015); Seidman, Fazlyab, Preciado and Pappas, Robust Deep Learning as Optimal Control: Insights and Convergence Guarantees (PMLR, 2020).

17. Where can a learned model go?

Question. You learn a neural network model of a system and ask where it can take a set of starting states in k steps. How does a cheap guaranteed bound compare with Monte Carlo sampling, which is accurate but guarantees nothing?

Try first. Fit an MLP to a simulated two-dimensional system, such as the Van der Pol oscillator. Push a box of starting states forward k steps by sampling many points (an inside estimate) and by interval bound propagation through the network's layers (an outside bound). Measure how much larger the bound is than the sampled set as k and the network's width grow.

Go further. Bound the k-step map in one shot rather than step by step, the idea of the first paper below, and compare tightness.

Read. Chen, Preciado and Fazlyab, One-Shot Reachability Analysis of Neural Network Dynamical Systems (ICRA, 2023); Fazlyab, Morari and Pappas, Safety Verification and Robustness Analysis of Neural Networks via Quadratic Constraints and Semidefinite Programming (arXiv:1903.01287).

18. Optimizers as dynamical systems

Question. Gradient descent, heavy-ball momentum, Nesterov's method and Adam are dynamical systems whose state is the parameter vector. How does their speed depend on how badly conditioned the problem is?

Try first. Run ridge and logistic regression on UCI datasets, controlling the condition number through feature scaling or the ridge penalty (Section 5B). Plot iterations to a fixed tolerance against the condition number κ, and compare with the textbook rates for quadratics: of order κ for gradient descent and $\sqrt{\kappa}$ for the accelerated methods. Section 2B's ball on a contour map is the picture to start from.

Go further. Compute guaranteed rates with the small semidefinite programs of the integral quadratic constraint papers below, solved in CVXPY, and compare them with your measured rates.

Read. Lessard, Recht and Packard, Analysis and Design of Optimization Algorithms via Integral Quadratic Constraints (SIAM J. Optim., 2016); Fazlyab, Ribeiro, Morari and Preciado, Analysis of Optimization Algorithms via Integral Quadratic Constraints: Non-Strongly Convex Problems (SIAM J. Optim., 2018); Seidman, Fazlyab, Preciado and Pappas, A Control-Theoretic Approach to Analysis and Tuning of Douglas-Rachford Splitting (IEEE L-CSS, 2020).

19. Learning together over a network

Question. When the data sits on many machines that can only talk to their neighbors, how does the shape of the communication network slow learning down?

Try first. Split a public classification dataset across 20 to 50 simulated agents connected as a ring, a grid, a random graph and a star. In decentralized gradient descent each agent averages its neighbors' parameters, then takes a local gradient step on logistic regression. Plot iterations to reach the centralized accuracy against the graph's algebraic connectivity, the Laplacian eigenvalue that sets how fast averaging spreads.

Go further. Give each agent mostly one class, and find the few added edges that speed learning up the most.

Read. Seidman, Fazlyab, Pappas and Preciado, Chebyshev-Accelerated Primal-Dual Method for Distributed Optimization (CDC, 2018); Paternain, Fazlyab, Preciado and Ribeiro, A Prediction-Correction Primal-Dual Algorithm for Distributed Optimization (ACC, 2019); Preciado and Zavlanos, Distributed Network Design for Laplacian Eigenvalue Placement (IEEE TCNS, 2017).

Theme 4: The course's methods under the microscope

These topics need no new field: they test claims the course makes, on data where you can check them. They suit a group that wants a clean, carefully validated study.

20. Do the probabilities mean what they say?

Question. The cost-optimal threshold of Section 3D, c_FP / (c_FP + c_FN), assumes the predicted probabilities are calibrated. How calibrated are logistic regression, random forests, gradient boosting and neural networks out of the box, and how much does

recalibrating save?

Try first. Use two or three binary datasets from the UCI repository, such as Adult or Bank Marketing. Draw reliability diagrams on held-out data, recalibrate with Platt scaling and isotonic regression on a separate split (scikit-learn's guide), and compare the expected cost at the cost-optimal threshold for several cost ratios, with bootstrap intervals.

Go further. Check that recalibration leaves the ROC curve unchanged, and whether the NLL, Section 3E's proper score, ranks the models the way the cost does.

Read. Guo, Pleiss, Sun and Weinberger, On Calibration of Modern Neural Networks (ICML, 2017).

21. A rare positive class

Question. When positives are rare, which repair works best: resampling to balance the classes, weighting the classes in the loss, or simply moving the threshold?

Try first. Use the credit-card fraud data on OpenML, where fraud is well under 1% of transactions. With stratified folds (Section 4B), compare logistic regression and gradient boosting with no repair, undersampling, oversampling, class weights and a moved threshold. Score with precision-recall curves, average precision and the expected cost at a stated cost ratio, never accuracy.

Go further. Show that undersampling a logistic model mostly shifts its intercept, so it acts as a threshold move in disguise.

Read. Saito and Rehmsmeier, The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets (PLoS ONE, 2015).

22. Honest error bars

Question. Cross-validation and the bootstrap each give an error estimate with an interval. How often do those intervals contain the true test error, and which error does cross-validation actually estimate?

Try first. Simulate data from a known Nature, as Sections 2B and 4A do, so the true error of every fitted model can be computed on a large fresh sample. For logistic regression and kNN, compute 10-fold CV with its naive standard error and bootstrap percentile intervals for the error rate and the AUC. Measure coverage over 1,000 datasets at N = 50, 200 and 1,000.

Go further. Repeat on a real dataset, with a large held-out set standing in for the truth, and try the nested cross-validation of the paper below.

Read. Bates, Hastie and Tibshirani, Cross-validation: what does it estimate and how well does it do it? (JASA, 2024).

23. Trees or neural networks for tables?

Question. Gradient-boosted trees often beat neural networks on tabular data. Does that hold on datasets you pick, and which property of the data explains it?

Try first. Take four to six datasets from the tabular benchmark of the paper below, hosted on OpenML. Compare a random forest, gradient boosting and an MLP with the same tuning budget for each (for example, 30 random-search trials scored by inner cross-validation). Then add uninformative inputs, or rotate the inputs at random, and see which model suffers.

Go further. Learning curves: does the MLP catch up as N grows?

Read. Grinsztajn, Oyallon and Varoquaux, Why do tree-based models still outperform deep learning on typical tabular data? (NeurIPS, 2022).

24. Double descent

Question. Section 2B teaches that test error rises once a model is flexible enough to overfit; Section 8B shows it can fall again past the point where the model fits the training data exactly. When does the second descent appear, and does a ridge penalty remove the peak?

Try first. Map the inputs through m random ReLU or Fourier features and fit minimum-norm least squares, on synthetic data with a few dozen inputs and on a subset of MNIST. Sweep m through N, the point where the model starts to interpolate, over several seeds and noise levels; then add a small ridge penalty and repeat. Use many inputs: with a single input the second descent can fail to appear.

Go further. Relate the peak to the smallest singular values of the feature matrix.

Read. Belkin, Hsu, Ma and Mandal, Reconciling modern machine-learning practice and the classical bias-variance trade-off (PNAS, 2019).

25. Which inputs really matter?

Question. The lasso, best subset selection and random-forest importance each name the inputs that matter. How stable are those lists when the data change a little, and do they find the true inputs?

Try first. Simulate a few true inputs among many correlated distractors, so the answer key is known, then repeat on a real dataset with many inputs from UCI or OpenML. Bootstrap the data 200 times and record how often each method selects each input (Sections 1C, 5A, 5B and 7B), scoring against the truth with precision and recall.

Go further. Stability selection, which keeps only the inputs chosen in most subsamples, and the elastic net on groups of correlated inputs.

Read. Meinshausen and Bühlmann, Stability Selection (J. R. Stat. Soc. B, 2010).

Your proposal

Your proposal fits on one page. Copy this template into a plain-text file and fill in every field. A field you cannot fill yet starts with "Open:" followed by your question, and the feedback will answer it.

```
# [Title: the question, in under 12 words]

## Group
[Your Canvas group number]

## Topic
[Number from this guide, or "own"]

## Question
[One sentence, ending in a question mark.]

## Why it matters
[Two or three sentences: who would use the answer, and how.]

## Data
[A public link, and what you found when you loaded it: N and the number of
inputs. For a simulation: the model and its true parameters.]

## Baseline
[A method from the course, and how you will tune it.]
```

```
## Comparison
[One or two methods you will set against the baseline.]

## Evaluation
[The metric, the validation scheme (folds, time order, test set), and how you
will put error bars on it.]

## Key figure
[The one figure that would answer the question: its axes, and what each curve
or bar shows.]

## Risks and fallback
[What could fail, and the smaller paper you would write instead.]

## Who does what
[One line per member, first names only.]

## AI plan
[Which tools, for which steps.]
```

Submitting your proposal

Submit one plain-text file per group, written in the template above, so that every proposal can be read, checked and answered quickly.

1. Save the filled template as a plain-text file named `proposal.txt`. No other format is accepted: not `.md`, PDF or Word.
2. Keep every `##` heading exactly as written and in the same order, and write only under the headings. A proposal that departs from the template gets less specific feedback.
3. Keep it under 600 words, not counting the headings.
4. Leave out full names, Penn IDs and email addresses: Canvas already records who submitted. First names under "Who does what" are fine.
5. Before you submit, open your data link, load the data, and report what you found under "Data".
6. The member whose last name comes first in alphabetical order uploads the file to the "Final Project: Proposal" assignment on Canvas for the whole group, by 11:59 PM Eastern time on `Nov 2, 2026`.

What happens next. You will receive written feedback as a comment on your Canvas submission. It checks your proposal against the four points in "What makes a topic work" (a sharp question, something to check against, a course baseline, honest error bars), answers your "Open:" questions, and suggests your first two or three steps.

Before you submit the final paper

Every box below should be ticked before the paper goes in.

- [ ] The question appears in the abstract and is answered in the conclusion.
- [ ] Every result carries an error bar, and every comparison says whether the difference exceeds it.
- [ ] The test set was used once, after every choice was made.
- [ ] Every step learned from data sits inside the folds.
- [ ] A course baseline appears in every results table.
- [ ] A teammate reran the code from scratch with the stated seeds and got the same numbers.
- [ ] A member of the group opened and read every reference.
- [ ] Every figure has labeled axes with units, and a caption that states its point.
- [ ] The paper follows the project format, once it is released.
- [ ] The acknowledgements include the AI use statement.

Data, code and reading

Every source below is public and downloadable without scraping. Read a dataset's license and documentation before you build on it, and cite it in the paper.

Source	What it holds	Topics
SNAP	Large social, communication and collaboration networks	3, 6, 7
Netzschleuder	Hundreds of networks with metadata, from many fields	2, 8
SocioPatterns	Face-to-face contact networks in schools, hospitals and offices	1, 5

Source	What it holds	Topics
Cornell hypergraph data	Group interactions: contacts, email threads, co-authorship	8
Our World in Data COVID-19	Cases, deaths and testing by country	4
Delphi Epidata	US influenza-like illness and other surveillance signals	4
NYC TLC trip records	Every taxi and ride-hail trip in New York City, by month	13
PhysioNet EEG Motor Movement/Imagery	64-channel EEG from 109 subjects	11
PDEBench	Simulated solutions of standard PDEs	12
UCI repository	Classic tabular datasets	18, 19, 20, 22, 25
OpenML	Tabular datasets and benchmark suites	21, 23, 25

MNIST and Fashion-MNIST (topics 16, 19 and 24) come built into PyTorch and Keras.

Libraries worth knowing: scikit-learn for everything in the course, NetworkX for graphs, CVXPY for convex and geometric programs, PyDMD, PyKoopman and PySINDy for dynamics, PyTorch Geometric for graph neural networks, Gymnasium for control environments, and LipSDP for Lipschitz bounds.